

9 years after Java 9

What do we do with the Module System?

A celebratory graphic for Star Trek's 60th anniversary. The background is a deep space scene with a blue and purple nebula and numerous stars. The text "STAR TREK 60" is centered in a bold, metallic, italicized font. The entire graphic is framed by a stylized border consisting of blue and purple lines and shapes, including a grid of small squares in the top-left corner and diagonal lines in the top-right corner.

STAR TREK 60

What does the Java Module System (JPMS) give us?

An Example: Starfleet Database

How do we use it in the real world?

Learning from real-world projects

Starfleet Database

Search

kirk

Results

Aurelan Kirk

> George Samuel Kirk

Jackson Roykirk

James T. Kirk

George Samuel Kirk (Lieutenant)

Series (2)

Star Trek: The Original Series

Star Trek: Strange New Worlds

Episodes (6)

Operation -- Annihilate! (TOS - S1E29)

Strange New Worlds (SNW - S1E1)

Children of the Comet (SNW - S1E2)

Lift Us Where Suffering Cannot Reach (SNW - S1E6)

All Those Who Wander (SNW - S1E9)

A Quality of Mercy (SNW - S1E10)

Press 'ESC' to quit

An Example: Starfleet Database

<https://github.com/jjohannes/starfleet-database>

```
starfleet-db-model/src/main/java
├─ module-info.java
└─ starfleet/db/model/*.java
```

```
starfleet-db-data/src/main/java
├─ module-info.java
├─ starfleet/db/data/*.java
└─ starfleet/db/data/internal/model/*.java
```

```
starfleet-db-layout/src/main/java
├─ module-info.java
├─ starfleet/db/layout/*.java
└─ starfleet/db/layout/internal/*.java
```

```
starfleet-db-model/src/main/java/module-info.java
```

```
module starfleet.db.model {           // Module Name
    exports starfleet.db.model;       // Public package
}
```

```
starfleet-db-data/src/main/java/module-info.java
```

```
module starfleet.db.data {
    exports starfleet.db.data; // 'internal.model' package hidden

    requires transitive starfleet.db.model; // My architecture in Java
    requires java.net.http;                // JDK module
    requires tools.jackson.databind;        // 3rd party module (Jar)

    opens starfleet.db.data.internal.model to tools.jackson.databind;
}
```

```
starfleet-db-layout/src/main/java/module-info.java
```

```
module starfleet.db.layout {
    exports starfleet.db.layout;

    requires transitive starfleet.db.model;
    requires dev.tamboui.toolkit;
}
```

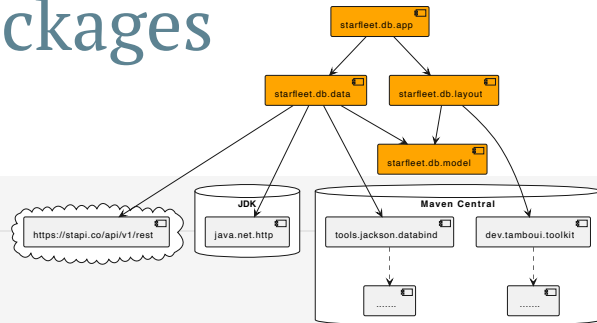
Smart compiler: module-private packages

When a class is *public* in a *private* package in a module...

```
starfleet/db/data/internal/model/Episode.java
```

```
package starfleet.db.data.internal.model;
```

```
public record Episode(String title, Series series) { }
```



...the compiler fails when trying to access that in another module:

```
StarfleetDatabase.java:3: error: package starfleet.db.data.internal.model is not visible
```

```
import starfleet.db.data.internal.model.*;
```

^

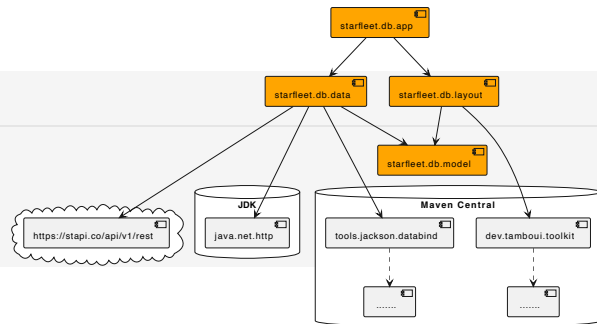
```
(package starfleet.db.data.internal.model is declared in module starfleet.db.data, which does not export it)
```

Smart compiler: less ClassNotFoundException

When a module is *required* by Module Name...

starfleet-db-data/src/main/java/module-info.java

```
module starfleet.db.data {
  requires tools.jackson.databind;
}
```



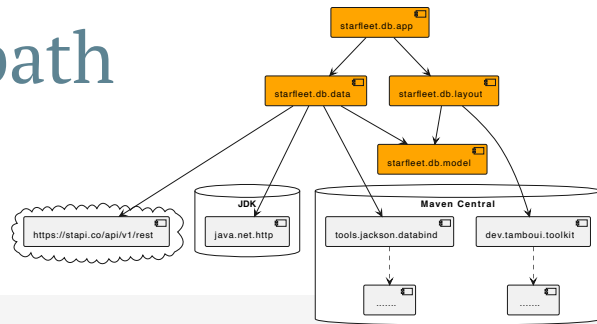
...the compiler fails to compile the `module-info.java` if not present:

```
starfleet-db-data/src/main/java/module-info.java:6: error: module not found: tools.jackson.databind
    requires tools.jackson.databind;
```

Smart runtime: no corrupted classpath

All Jars, including 3rd-party Jars, have a `module-info.class`

Missing transitive requirements are detected at startup



```
java --module-path lib --module starfleet.db.app
```

Error occurred during initialization of boot layer

```
java.lang.module.FindException: Module tools.jackson.core not found, required by tools.jackson.databind
```

Duplication is detected at startup

```
java --module-path lib --module starfleet.db.app
```

Error occurred during initialization of boot layer

```
java.lang.module.FindException: Two versions of module tools.jackson.databind found in lib  
(jackson-databind-3.2.1.jar and jackson-databind-3.2.2.jar)
```

Smart runtime: controlled reflection

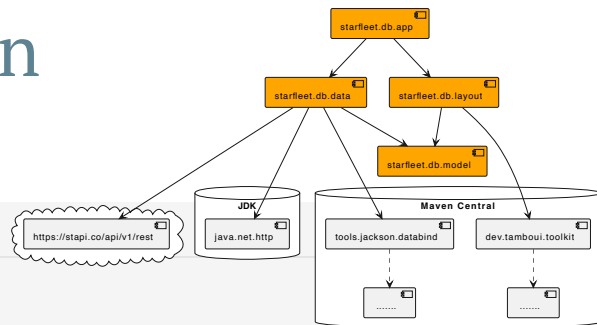
Reflective access to internal packages needs to be allowed explicitly

starfleet-db-data/src/main/java/module-info.java

```
module starfleet.db.data {  
    // ...  
    requires tools.jackson.databind;  
    // opens starfleet.db.data.internal.model to tools.jackson.databind;  
}
```

```
java --module-path lib --module starfleet.db.app
```

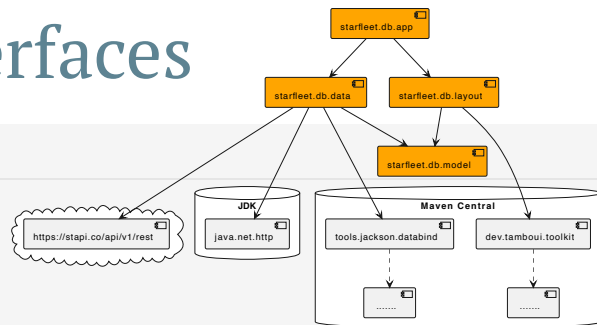
```
IllegalAccessException: access to public member failed: starfleet.db.data.internal.model.Character.<init>,  
    from class tools.jackson.databind.introspect.AnnotatedConstructor$InvokerHolder (module tools.jackson.databind)
```



Smart runtime: cleaner service interfaces

tamboui-core.jar/module-info.class

```
module dev.tamboui.core {  
    // ...  
    uses dev.tamboui.terminal.BackendProvider;  
}
```



tamboui-jline3-backend.jar/module-info.class

```
module dev.tamboui.jline_three.backend {  
    // ...  
    provides dev.tamboui.terminal.BackendProvider with dev.tamboui.backend.jline3.JLineBackendProvider;  
}
```

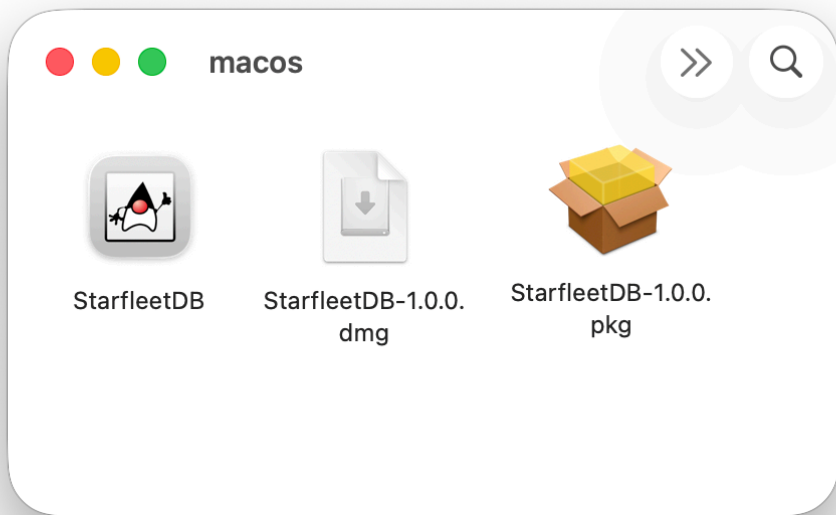
Use the interface without declaration – `ServiceLoader.load(BackendProvider.class)`

```
ServiceConfigurationError: dev.tamboui.terminal.BackendProvider: module starfleet.db.layout does not declare 'uses'
```

More JDK tools

`jlink` / `jpackage` to assemble a complete installable package (JDK module included)

```
jpackage --module starfleet.db.app --module-path lib --type dmg
```



What does the Java Module System (JPMS) give us?

An Example: Starfleet Database

How do we use it in the real world?

Learning from real-world projects

My Java Module System (JPMS) journey

- **2017** - Java Module System released with Java 9*
- **2020** - First release of JPMS support in Gradle 6.4
 - 2021 - stable release in Gradle 7.0
- **2020** - First release of `org.gradle.extra-java-module-info` plugin
- **2021** - LION project (<https://koerber-pharma.com>) introduces JPMS with Gradle
- **2021** - First release of `org.gradle.java-module-dependencies` plugin (initially for use in LION)
- **2022** - `1.0` releases of `org.gradle.*java-module*` plugins
- **2023** - Hiero Ledger project (<https://hiero.org>) introduces JPMS with Gradle
- **2025** - JabRef project (<https://jabref.org>) streamline their JPMS/Gradle setup with GradleX plugins
- **2022–2026** plugins evolve, tutorials on <https://javarca.de> and <https://www.youtube.com/@jjohannes>

*I wasn't involved here, but Gradle introduced the *implementation* scope in Gradle 3.4 at the same time (we'll get to this)

Why is it hard to introduce JPMS?

5 Reasons why

Reason 1 (of 5) why it is hard to use JPMS

Ecosystem: deal with redefinition of what a JAR is and what a package is (add `module-info.class`)

Common issues

- Java 8 backward-compatibility (e.g. JUnit)
- Split-package issue (e.g. Testcontainers)
- Split-package issue with resources (e.g. Besu)
- Reflection / services loading (e.g. LangChain4j)

Popular libraries adding `module-info.class` ?

- 07/23 - Apache Commons
- 09/24 - Netty
- 03/25 - Guava
- ??/?? – Spring (issue)
- ??/?? – Testconainers (split package)

In a recent survey of the 1,000 most popular artifacts on Maven Central, just 232 had explicit module definitions

Source: Netflix

Solution: temporarily patch Jars for your project

org.gradle.extra-java-module-info

```
extraJavaModuleInfo {
    // add 'module-info.class'
    module("org.jline:jline", "org.jline") {
        requires("java.logging")
    }
    // merge Jars
    module("org.testcontainers:testcontainers",
        "org.testcontainers") {
        mergeJar("org.testcontainers:kafka")
    }
}
```

- Patch rules example: Hiero projects
- Patch rules example: JabRef project

Reason 2 (of 5) why it is hard to use JPMS

Ecosystem: metadata is *weak* for many libraries (even without `moudle-info.class`)

```
https://repo1.maven.org/maven2/ # Maven Central
└─ org/apache/logging/log4j/log4j-core/2.26.1/
    │   log4j-core-2.26.1.jar
    │   └─ org/apache/**/*.java
    │       └─ module-info.class # Java module-info
    └─ log4j-core-2.26.1.pom # POM Metadata
        └─ log4j-core-2.26.1.module # Gradle Metadata
```

Common issues

- Transitive dependencies that are not needed
- Transitive unnecessarily visible at compile time
- Alternative variants (classifiers) difficult to use
- Version alignment for multiple Jars

Popular libraries with Gradle Metadata

- JUnit / Jackson / Guava / Log4j

Solution: patch metadata for your project

```
org.gradle.extra-java-module-info

jvmDependencyConflicts.patch {
    // add Jars with classifier
    module("io.netty:netty-transport-native-epoll") {
        addFeature("linux-x86_64")
        addFeature("linux-aarch_64")
    }
    // remove (exclude) transitive dependencies
    module("com.google.guava:guava") {
        removeDependency("com.google.code.findbugs:jsr305")
        removeDependency("org.checkerframework:checker-qual")
    }
}
```

- Patch rules example: Hiero projects
- Patch rules example: JabRef project

Reason 3 (of 5) why it is hard to use JPMS

Dependency management in projects is often partially broken

1. No clear dependency definition process (who adds dependencies how in which file with which notation)
2. No central version management for 3rd party libraries
3. Dependency management *buried* in other build configuration (`build.gradle` / `pom.xml` mix concerns)
4. No central dependency conflict management (e.g. *excludes* scattered over `build.gradle` / `pom.xml`)
5. No tooling that helps to ensure consistency
6. No 3rd party library update process



Recipe for Carefree Dependency Management

1. Decide on a method and define Dependencies between Modules in dedicated files

 Java Module System: ... `<module-folder>/src/main/java/module-info.java`
 Gradle: ... `<module-folder>/build.gradle.kts`
 Maven: ... `<module-folder>/pom.xml`

2. Version management in a dedicated file

 Gradle: ... `<bom-folder>/build.gradle.kts`
 Maven: ... `<bom-folder>/pom.xml`

❌ Do not mix with 1.

3. Configure each build concern in a dedicated file

 Gradle: ... `<plugins-folder>/<build-concern>.gradle.kts` (plugin composition)
 Maven: ... `<configs-folder>/<build-concern>/pom.xml` (parent pom hierarchy)

❌ Do not mix with 1.

4. Conflict management in a dedicated file

 Gradle: ... `<plugins-folder>/dependency-rules.gradle.kts`
 Maven: ... `<configs-folder>/dependency-rules/pom.xml`

❌ Do not mix with 1.

5. Configure dependency analysis and make it a regular check

 Gradle: ... `<plugins-folder>/quality-check.gradle.kts`
 Maven: ... `<configs-folder>/quality-check/pom.xml`

6. Configure Renovate or Dependabot for automatic update PRs

 Renovate: ... `renovate.json`
 Dependabot: ... `github/dependabot.yml`

Reason 4 (of 5) why it is hard to use JPMS

Insufficient tools support / dependency management is an *often forgotten* feature of "build" tools

- *Responsibilities*: Java team (Oracle), Maven Central (Sonatype), Maven (Apache), Gradle (Gradle Inc), IntelliJ (Jetbrains), Android Studio (Google), Eclipse (Eclipse Foundation)
- *Gradle core (dependency management)*
 - 2017 / Gradle 4 – `implementation` / `api` scopes (mirrors `requires` / `requires transitive`)
 - 2018 / Gradle 5 – Dependency Constraints, BOM support, Component Metadata Rules
 - 2019 / Gradle 6 - Variant-aware dependency management, Gradle Metadata format
 - 2021 / Gradle 7 - JPMS support
 - 2026 / Gradle 9 – improvements since Gradle 7 in all areas
- *GradleX plugins*
 - 2022 – `1.0` releases of JPMS plugins
 - 2024 – `2.0` release of `jvm-dependency-conflict-resolution` plugin

Reason 5 (of 5) testing in Java is very "whitebox"

Test setup likely needs to be changed for full JPMS setups

By default, tests are in a separate module

```
starfleet-db-data
├─ src/main/java
│   └─ module-info.java # starfleet.db.data
└─ src/test/java
    └─ module-info.java # starfleet.db.data.test
        └─ starfleet/db/data/test/**/*.java
```

```
open module starfleet.db.data.test {
    requires org.junit.jupiter.api;
    requires starfleet.db.data;
}
```

Testing internals: targeted package export

```
module starfleet.db.data {
    exports starfleet.db.data.internal.model
    to starfleet.db.data.test;
}
```

Testing internals: patch tests into main module

```
starfleet-db-data
├─ src/main/java
│   └─ module-info.java # starfleet.db.data
└─ src/test/java
    └─ # no module-info.java
        └─ starfleet/db/data/**/*.java
└─ build.gradle # for defining test 'requires'
```

build.gradle (with org.gradle.java-module-testing)

```
testModuleInfo {
    requires("org.junit.jupiter.api")
    requires("starfleet.db.data")
}
```

Why is it hard to introduce JPMS?

We could also ask...

Why is it hard to manage dependencies?

- Reason 1: strict package/module separation

Clean separation and no overlap may be good even without JPMS

- Reason 2: libraries with insufficient metadata

Missing module-info adds to this, but problem already exists (pom / module files)

- Reason 3: partially broken dependency management setups

Risks and future costs are rising independent of the use of JPMS

- Reason 4: insufficient tool support

Dependency management features introduced by Gradle in the past years are underused / missing guidelines

- Reason 5: whitebox testing setup

Successes (?) Stories

Hard to judge JPMS in isolation

- Important is: intentional modularization
- Important is: good and continuous tool support

Contra JPMS / disappointments

- Better tool support was expected
 - Improved over the past five years
 - Expert support was/is necessary
- Harder to work with *Modul Names* than with *group:artifact coordinates*
- `module-info.java` format is "foreign" to many developers despite being `.java`

We're excited to announce a preview of *ja* and its family of composable tools, that build on the capabilities of the Java Module System to provide a modern command line development experience for Java.

Leave the Class Path in the Rearview Mirror, Netflix, 19.09.2006

Pro JPMS

- *private* packages directly in Java
- Fewer risks and less technical dept through miss-configured classpath
- Code quality improved through modularization (would be possible without JPMS and Gradle only)
- Caching in build tool, enabled through modules, is an important productivity aspect
- Would recommend JPMS for other projects (with the now improved tool support)

Thank You



javarca.de



Jendrik Johannes



High quality Gradle plugins

gradlex.org



Understanding Gradle Video

[youtube.com/
@jjohannes](https://youtube.com/@jjohannes)



Gradle Training and Consulting

onepiece.software



Better Test Reporting on GitHub

testlens.app